

Chat Context Export

Platform: ChatGPT

Exported: 13/09/2026, 15:22:38

Conversation ID: 6a9d9ba7-a674-83e9-81f4-fbe3983136a4

Conversation

Assistant

Yes — this output is actually very useful. Your Mac is definitely capturing traffic successfully. The problem is now inside your processing path, not CMake or libpcap capture. The key difference is: while your application reported: is , meaning your libpcap handle is giving your application packets starting at the IP header, whereas is exposing Apple's PKTAP link type. Do this next Don't use yet. Let's isolate the problem with the actual Wi-Fi interface. First: Find your Wi-Fi interface — usually on macOS. Then run: And in another terminal: If your engine still prints only: with no packet/ClientHello output, then we know the problem is definitely in your packet callback / link-layer demux / TCP reassembly, not the interface. One important thing from your output Your output shows plenty of existing TCP/443 connections: But those are already-established connections. Your JA3 engine needs to see the initial ClientHello, which happens at the beginning of a TCP connection. So use a fresh connection: The is useful here because we're specifically testing TCP/TLS rather than HTTP/3/QUIC. And I think I see the next place we should inspect Because your engine says: I want to see exactly how you handles the datalink type. Please paste the current , particularly: and the code around: plus your link-layer demultiplexing logic. Don't change CMake or the parser yet. Your capture is working; we should trace: Your 51/51 tests already prove the parser/JA3 side is working. Now we need to find where the live packet path is dropping the packets.

Code Block 1 (text)

```
tcpdump:  
link-type PKTAP (Apple DLT_PKTAP)
```

Code Block 2 (text)

```
Datalink type: 12
```

Code Block 3 (text)

```
sudo tcpdump -D
```

Code Block 4 (text)

```
sudo ./build/tlsfp_engine -i en0 -f "tcp port 443" -v
```

Code Block 5 (text)

```
curl -v https://example.com
```

Code Block 6 (text)

```
[*] Listening on interface: en0  
[*] Capture initialized...
```

Code Block 7 (text)

```
10.7.15.147.53059 > 4.150.223.115.443
```

Code Block 8 (text)

```
curl -v --http1.1 https://example.com
```

Code Block 9 (text)

```
Datalink type: 12
```

Code Block 10 (text)

```
pcap_datalink(...)
```

Code Block 11 (text)

```
packet_callback(...)
```

Code Block 12 (text)

```
libpcap  
☒  
packet_callback  
☒  
DLT_RAW / link-layer demux  
☒  
IP  
☒  
TCP  
☒  
TCP reassembly  
☒  
TLS ClientHello  
☒  
parse_client_hello()  
☒  
JA3
```

```
(base) ❯ build git:(main) ❯ sudo tcpdump -i en0 -n "port 443" tcpdump: verbose output suppressed, use -
v[v]... for full protocol decode listening on en0, link-type EN10MB (Ethernet), snapshot length 524288
bytes 23:03:03.381336 IP 10.7.15.147.57987 > 172.64.155.209.443: Flags [.] , ack 174986408, win 23345,
length 0 23:03:03.414788 IP 172.64.155.209.443 > 10.7.15.147.57987: Flags [.] , ack 1, win 339, options
[nop,nop,TS val 933165121 ecr 1017321105], length 0 23:03:04.599848 IP 10.7.15.147.53014 >
52.8.172.150.443: Flags [P.] , seq 1477300077:1477300115, ack 30671848, win 2048, options [nop,nop,TS
val 1672537920 ecr 2380514142], length 38 23:03:04.862338 IP 52.8.172.150.443 > 10.7.15.147.53014:
Flags [.] , ack 38, win 179, options [nop,nop,TS val 2380528995 ecr 1672537920], length 0 (base) ❯ cpp git:
(main) ❯ sudo ./build/tlsfp_engine -i en0 -f "tcp port 443" -v [*] Listening on interface: en0 [*] Capture
initialized. Datalink type: 1 [-] Warning: Redis database unavailable; fingerprints will be reported as
unknown. [+] Captured ClientHello | Flow: [10.7.15.147]:58098 -> [20.207.73.82]:443 | Reassembled Size:
1533 bytes ❯ [V-Version] 0x0303 ❯ [V-SNI] github.com ❯ [V-ALPN] h2 ❯ [V-Ciphers] Count: 20 [0x1302,
0x1303, 0x1301, 0xc02c, 0xc02b, 0xcca9, 0xc030, 0xc02f, 0xcca8, 0xc00a, 0xc009, 0xc014, 0xc013,
0x009d, 0x009c, 0x0035, 0x002f, 0xc008, 0xc012, 0x000a] ❯ [V-Extensions] Count: 13 [0x0000, 0x0017,
0xff01, 0x000a, 0x000b, 0x0010, 0x0005, 0x000d, 0x0012, 0x0033, 0x002d, 0x002b, 0x001b] ❯ [V-Curves]
Count: 5 [0x11ec, 0x001d, 0x0017, 0x0018, 0x0019] ❯ [V-SigAlgs] Count: 10 [0x0403, 0x0804, 0x0401,
0x0503, 0x0805, 0x0805, 0x0501, 0x0806, 0x0601, 0x0201] ❯ [V-PreHash-JA3]
771,4866-4867-4865-49196-49195-52393-49200-49199-52392-49162-49161-49172-49171-157-156-53-47-49
160-49170-10,0-23-65281-10-11-16-5-13-18-51-45-43-27,4588-29-23-24-25,0 ❯ [V-PreHash-
JA4b]000a,002f,0035,009c,009d,1301,1302,1303,c008,c009,c00a,c012,c013,c014,c02b,c02c,c02f,c030,cc
a8,cca9 ❯ [V-PreHash-
JA4c]0005,000a,000b,000d,0012,0017,001b,002b,002d,0033,ff01_0403,0804,0401,0503,0805,0805,0501,
0806,0601,0201 ❯ [SNI] github.com ❯ [JA3 String]
771,4866-4867-4865-49196-49195-52393-49200-49199-52392-49162-49161-49172-49171-157-156-53-47-49
160-49170-10,0-23-65281-10-11-16-5-13-18-51-45-43-27,4588-29-23-24-25,0 ❯ [JA3 Hash]
ecdf4f49dd59effc439639da29186671 ❯ [JA3 Match] <database unavailable> ❯ [JA4]
t13d2013h2_a09f3c656075_7f0f34a4126d ❯ [JA4 Match] <database unavailable> [+] Captured
ServerHello | Flow: [20.207.73.82]:443 -> [10.7.15.147]:58098 | Reassembled Size: 127 bytes ❯ [V-
WireVersion] 0x0303 ❯ [V-Negotiated] 0x0304 ❯ [V-Cipher] 0x1301 ❯ [V-ALPN] <none> ❯ [V-Extensions]
Count: 2 [0x002b, 0x0033] ❯ [V-PreHash-JA3S]771,4865,43-51 ❯ [V-PreHash-JA4S]002b,0033 ❯ [JA3S
String] 771,4865,43-51 ❯ [JA3S Hash] f4febc55ea12b31ae17cfb7e614afda8 ❯ [JA3S Match] <database
unavailable> ❯ [JA4S] t130200_1301_a56c5b993250 ❯ [JA4S Match] <database unavailable> [+]
Captured ClientHello | Flow: [10.7.15.147]:58100 -> [185.199.110.133]:443 | Reassembled Size: 1552
bytes ❯ [V-Version] 0x0303 ❯ [V-SNI] avatars.githubusercontent.com ❯ [V-ALPN] h2 ❯ [V-Ciphers] Count: 20
[0x1302, 0x1303, 0x1301, 0xc02c, 0xc02b, 0xcca9, 0xc030, 0xc02f, 0xcca8, 0xc00a, 0xc009, 0xc014,
0xc013, 0x009d, 0x009c, 0x0035, 0x002f, 0xc008, 0xc012, 0x000a] ❯ [V-Extensions] Count: 13 [0x0000,
0x0017, 0xff01, 0x000a, 0x000b, 0x0010, 0x0005, 0x000d, 0x0012, 0x0033, 0x002d, 0x002b, 0x001b] ❯ [V-
Curves] Count: 5 [0x11ec, 0x001d, 0x0017, 0x0018, 0x0019] ❯ [V-SigAlgs] Count: 10 [0x0403, 0x0804,
0x0401, 0x0503, 0x0805, 0x0805, 0x0501, 0x0806, 0x0601, 0x0201] ❯ [V-PreHash-JA3]
```

771,4866-4867-4865-49196-49195-52393-49200-49199-52392-49162-49161-49172-49171-157-156-53-47-49
160-49170-10,0-23-65281-10-11-16-5-13-18-51-45-43-27,4588-29-23-24-25,0 [V-PreHash-
JA4b]000a,002f,0035,009c,009d,1301,1302,1303,c008,c009,c00a,c012,c013,c014,c02b,c02c,c02f,c030,cc
a8,cca9 [V-PreHash-
JA4c]0005,000a,000b,000d,0012,0017,001b,002b,002d,0033,ff01_0403,0804,0401,0503,0805,0805,0501,
0806,0601,0201 [SNI] avatars.githubusercontent.com

Assistant

Yes — this confirms your live capture path is working correctly. You have now proven: And you're successfully seeing real traffic: including: SNI: 20 cipher suites 13 extensions 5 supported groups 10 signature algorithms JA3 JA3S JA4 JA4S So do not change the CMakeLists.txt. One thing I would investigate next Your output shows: = 4588, which is a GREASE value. That means your live parser is apparently not filtering this particular GREASE supported-group value, despite the parser-level GREASE tests passing. This is especially interesting because earlier your parser has: So either: isn't recognized by your current , or the verbose output is displaying the raw parsed data from somewhere else, or you're running a stale/different build/source than the tests. And your JA3 currently contains: which means 4588 is actually entering the JA3 string. That's worth fixing because this is exactly the kind of real-world GREASE case your new tests are supposed to catch. Check Please paste your current: function from . I suspect the issue is that the implementation recognizes the older/common values like , , etc., but not all RFC 8701 GREASE values. The standard GREASE pattern is: where the high and low bytes have the same high nibble and the low nibble is . For example: But 4588 = , which does not match that pattern, so I would not immediately assume 4588 is GREASE. We should verify what that group actually is before changing anything. Your current output is therefore a good sign: the capture engine is functioning, and we've reached a genuine fingerprint/data-validation issue rather than an infrastructure issue.

Code Block 1 (text)

```
macOS en0
[
  libpcap DLT_EN10MB (1)
  [
    packet_callback
    [
      TCP reassembly
      [
        TLS ClientHello
        [
          parse_client_hello()
          [
            JA3 / JA4
```

Code Block 2 (text)

[+] Captured ClientHello | ... -> ...:443

Code Block 3 (text)

[V-Curves] Count: 5 [0x11ec, 0x001d, 0x0017, 0x0018, 0x0019]

Code Block 4 (text)

```
if (!is_grease(group)) {  
    out.supported_groups.push_back(group);  
}
```

Code Block 5 (text)

4588-29-23-24-25

Code Block 6 (text)

is_grease(...)

Code Block 7 (text)

0x?A?A

Code Block 8 (text)

0x0a0a
0x1a1a
0x2a2a
...
0xfafa

User

tell me that why is any not working it should give me more packets then ethernet obv

Assistant

You're right about the expectation, but there's an important macOS-specific detail: does not mean "Wi-Fi + Ethernet, and therefore more packets." On macOS, is a pseudo-interface implemented by libpcap using Apple's PKTAP mechanism. It is not equivalent to Linux's interface. Your own outputs show this: while: So and are going through different capture paths. Why you're seeing this On macOS: Whereas: libpcap then reports: for . Your application, however, reported: which is: That is the critical difference. Your code is presumably doing something along the lines of: and your demux knows how to process Ethernet: but your capture is giving you a different link-layer format. So why did work? Because: and your code already

handles that correctly. With: your code isn't necessarily interpreting the packet correctly. And there's another important point You said: any should give me more packets than ethernet obviously Conceptually, yes, if you're trying to capture traffic across multiple physical/logical interfaces. But on your Mac, you're probably only using Wi-Fi right now. Your machine has things like: But not all of these should necessarily be treated as ordinary Ethernet traffic, and some traffic may be duplicated or represented differently. So: doesn't guarantee: And for a TLS fingerprinting engine, you actually don't want to blindly count everything. You want to correctly decode every relevant interface. The real fix for your project If your project requirement is: Capture TLS traffic regardless of whether it comes through Wi-Fi, Ethernet, VPN, etc. then your capture layer should support the relevant DLTs. At minimum, you've demonstrated: The interesting question is therefore: Why is your application getting DLT_RAW when reports DLT_PKTAP? That's something I would investigate before changing anything. Run this: and then, in another terminal: Your application says: while tcpdump says: If both are genuinely using the same libpcap installation, we should inspect how your opens the interface and handles the datalink type. So don't modify CMake yet. The CMake configuration successfully links libpcap, and proves the capture path works. The next thing I want to see is your implementation in , especially the / / portion and how you obtain . That's where the discrepancy is likely coming from.

Code Block 1 (text)

```
tcpdump -i any  
link-type PKTAP (Apple DLT_PKTAP)
```

Code Block 2 (text)

```
tcpdump -i en0  
link-type EN10MB (Ethernet)
```

Code Block 3 (text)

```
en0  
❌ ❌ actual Wi-Fi interface  
❌ ❌ Ethernet-style frames
```

Code Block 4 (text)

```
any  
❌ ❌ macOS PKTAP pseudo-interface  
❌ ❌ system-wide packet metadata/capture path
```

Code Block 5 (text)

```
DLT_PKTAP
```

Code Block 6 (text)

Datalink type: 12

Code Block 7 (text)

```
DLT_RAW
```

Code Block 8 (text)

```
pcap_open_live(...)
⊞
pcap_datalink(...)
⊞
link-layer demux
```

Code Block 9 (text)

```
DLT_EN10MB = 1
```

Code Block 10 (text)

```
en0 ⊞ DLT_EN10MB ⊞ your Ethernet parser ⊞ IP ⊞ TCP ⊞ TLS
```

Code Block 11 (text)

```
any ⊞ macOS pseudo-interface ⊞ different packet representation
```

Code Block 12 (text)

```
en0    Wi-Fi
lo0     loopback
awdl0   AirDrop/Apple Wireless Direct Link
llw0    low-latency wireless
utunX   VPN/tunnel interfaces
...
```

Code Block 13 (text)

```
-i any
```

Code Block 14 (text)

```
packets(any) > packets(en0)
```

Code Block 15 (text)

```
DLT_EN10MB = 1    ⊞ en0 works
DLT_RAW   = 12    ⊞ your `any` application reported this
DLT_PKTAP      ⊞ macOS tcpdump says `any` uses this
```

Code Block 16 (text)

```
sudo ./build/tlsfp_engine -i any -f "tcp port 443" -v
```

Code Block 17 (text)

```
sudo tcpdump -i any -nn -c 1 "tcp port 443"
```

Code Block 18 (text)

```
Datalink type: 12
```

Code Block 19 (text)

```
link-type PKTAP
```